



Universidad Carlos III
Curso Sistemas Distribuidos 2025-26
Curso 2025-26
Práctica final

Fecha: **22/04/2026**

GRUPO: **82 & 84**

Alumnos: **Victor Sainz Mora 100495888 - 100495888@alumnos.uc3m.es /**
David Torres Paillacho 100522187 - 100522187@alumnos.uc3m.es

Tabla de Contenido

Introducción.....	2
Arquitectura del sistema.....	2
Descripción del código.....	3
Servidor:.....	3
Cliente:.....	4
Protocolo de comunicación:.....	5
Compilación y ejecución.....	5
Batería de pruebas.....	6
Pruebas con app_cliente.py.....	6
Problemas encontrados y conclusiones.....	7

Introducción

En esta práctica hemos hecho una aplicación distribuida para enviar mensajes entre usuarios. El sistema tiene dos partes: un servidor programado en C y un cliente programado en Python.

El servidor se encarga de guardar los usuarios registrados, saber cuáles están conectados, almacenar los mensajes que no se pueden entregar en ese momento y enviarlos cuando el usuario se vuelve a conectar. El cliente, por su parte, permite que el usuario use la aplicación escribiendo comandos por consola.

La comunicación entre el cliente y el servidor se hace usando sockets TCP. Para las operaciones principales se abre una conexión con el servidor. Además, cuando un usuario se conecta, el cliente crea un hilo que queda escuchando para recibir los mensajes que le puedan llegar.

En esta entrega se ha implementado la parte principal del servicio de mensajería: registrarse, darse de baja, conectarse, desconectarse, ver qué usuarios están conectados y enviar mensajes.

Además de estas operaciones básicas, la funcionalidad se ha ampliado para soportar el envío de archivos adjuntos junto con los mensajes de texto.

Para convertir esta aplicación en una arquitectura distribuida completa, hemos implementado dos tecnologías adicionales al sistema base. Por un lado, se ha implementado un servidor secundario de registro de actividad utilizando llamadas a procedimientos remotos con Sun RPC. Por otro lado, se ha incorporado un servicio web basado en el estándar SOAP, desarrollado en Python con Flask, que el servidor utiliza para normalizar y limpiar el texto de los mensajes antes de ser entregados. De esta forma, la práctica abarca el uso combinado de sockets TCP, RCP y servicios web.

Arquitectura del sistema

Para nuestro programa, hemos diseñado esta práctica dividiendo el trabajo en cuatro partes distintas que se comunican por red. De esta forma, cada servicio se encarga de una tarea específica.

La parte principal del sistema es el servidor central. Actúa como el coordinador de todo, que se encarga de gestionar qué usuarios están registrados, quiénes están conectados y de enrutar los mensajes. Para poder escuchar a varios clientes a la vez sin que el programa se quede bloqueado, hemos utilizado una arquitectura multi hilo con la librería pthreads.

Este servidor necesita de tres módulos fundamentales y principales para funcionar:

- **Clientes de mensajería:** este módulo se encarga de la interfaz con el usuario. Permite simular un cliente, escribiendo comandos por la consola para registrarse o mandar mensajes. Lo más importante es que tiene un hilo escuchando en segundo plano. Así si alguien nos manda un mensaje o un archivo, nos aparece en pantalla al instante aunque estemos escribiendo otra cosa.
- **Servidor de Logs:** en la parte de RPC nos encargamos de llevar el registro de todo lo que ocurre en el sistema. En lugar de que el servidor imprima todo por su pantalla cada vez que hay una operación importante, el servidor llama a este módulo mediante RPC para que sea él quien imprima el evento. Así separamos la lógica de los mensajes del registro de actividad
- **Servicio Web:** este módulo se encarga de procesar y limpiar el texto de los mensajes. Cuando el servidor central recibe un mensaje para enviar, se lo pasamos al servicio web por HTTP. El servicio limpia el texto, eliminando espacios en blanco, y se lo devuelve al servidor ya corregido para que el destinatario lo reciba limpio.

Al final, todo el sistema se mantiene en conjunto gracias a un protocolo que hemos diseñado sobre TCP. Al combinar sockets para los mensajes, RPC para los logs y SOAP para el servicio web, hemos conseguido que el sistema sea distribuido.

Descripción del código

Servidor:

El servidor está hecho en el fichero server.c. Su trabajo es recibir las peticiones de los clientes y guardar el estado general del sistema.

Cuando se inicia, el servidor crea un socket TCP, lo vincula al puerto que se pasa por línea de comandos y se queda esperando conexiones. Cada vez que llega una petición, crea un hilo para atenderla, así puede gestionar varios clientes a la vez.

También guarda una lista con los usuarios registrados. De cada usuario se almacena su nombre, si está conectado o no, su IP, el puerto por el que escucha el cliente y el último identificador de mensaje que se le ha asignado.

También tiene una lista de mensajes pendientes, donde se guardan los mensajes que no se han podido entregar porque el usuario destinatario no estaba conectado.

Las operaciones que se han implementado son:

REGISTER, UNREGISTER, CONNECT, DISCONNECT, USERS, SEND y SENDATTACH.

Como el servidor puede atender varias peticiones al mismo tiempo, se usan mutex para proteger las listas compartidas. Así se evitan problemas cuando varios hilos intentan leer o modificar los mismos datos a la vez.

Además de las operaciones básicas, este módulo se encarga de gestionar el envío de archivos mediante el comando **SENDATTACH**. En este caso, el servidor no solo enruta el texto del mensaje, sino que también coordina el envío de metadatos (nombre del fichero) para que el cliente receptor sepa que va a recibir un archivo adjunto.

Finalmente, para que el sistema sea realmente distribuido, el servidor central se encarga de comunicarse con los servicios externos:

- **Integración con RPC:** cada vez que un cliente realiza una operación, el servidor actúa como cliente RPC y llama a las funciones del servidor de logs, delegando así la responsabilidad de registrar la actividad.
- **Integración con SOAP:** cuando se recibe un mensaje o archivo, este módulo contacta con el servicio web mediante peticiones HTTP. Le envía el texto original y espera a recibir la versión procesada y limpiada antes de intentar entregársela al destinatario.

Cliente:

El cliente está hecho en el fichero client.py. Para ejecutarlo hay que indicar la IP y el puerto del servidor. Cuando arranca, aparece una consola donde el usuario puede escribir los comandos.

Los comandos disponibles son:

```
REGISTER <userName>
UNREGISTER <userName>
CONNECT <userName>
DISCONNECT <userName>
USERS
SEND <userName> <message>
SENDATTACH <userName> <fileName><message>
QUIT
```

Cuando el usuario usa **CONNECT**, el cliente abre un socket de escucha en un puerto libre y crea un hilo aparte. Ese hilo se queda esperando mensajes que pueda enviar el servidor. Así, el cliente puede recibir mensajes de otros usuarios mientras el programa principal sigue esperando comandos por teclado.

En resumen, el hilo principal lee lo que escribe el usuario y manda las peticiones al servidor. Por otro lado, el hilo secundario se encarga de recibir los mensajes entrantes, las confirmaciones de entrega y de procesar la llegada de archivos adjuntos sin interrumpir lo que el usuario esté tecleando.

Protocolo de comunicación:

El protocolo usado funciona con cadenas de texto que terminan en '\0', donde cada petición empieza con el nombre de la operación y después se manda el resto de datos.

Por ejemplo: para registrar un usuario se envía **REGISTER** y luego el nombre del usuario. Para mandar un mensaje se envía **SEND**, junto con el nombre del que lo envía, el destinatario y el texto del mensaje.

Las respuestas del servidor se envían con un byte. Si el valor es 0, significa que la operación ha ido bien, en caso de que el valor sea otro significa que indica algún error, como que el usuario no existe, que ya está registrado o que no está conectado.

Cuando el servidor tiene que entregar un mensaje a un cliente que está conectado, este se conecta al socket de escucha de ese cliente y le manda: la operación **SEND MESSAGE**, el nombre del remitente, el identificador del mensaje y el texto. Si todo sale bien, el servidor también avisa al usuario que envió el mensaje mediante **SEND MESS ACK**.

Por otro lado, para el envío de archivos, el proceso cambia un poco porque necesitamos manejar más datos de golpe. En lugar de un mensaje simple, cuando se lanza un **SENDATTACH**, el cliente envía primero el nombre de la operación y después va añadiendo en orden el remitente, el destinatario y el nombre del archivo original. Por último, se concatena el bloque de bytes con el contenido real del fichero para que llegue todo en el mismo flujo.

Cuando el servidor entrega este archivo al receptor, se conecta a su puerto de escucha y le envía la orden **SEND MESSAGE_ATTACH**. A continuación, le pasa el nombre del remitente, el nombre del archivo y el mensaje. De esta forma, el protocolo permite al cliente receptor saber exactamente qué campos va a leer y darse cuenta de que no es un mensaje normal.

Compilación y ejecución

El servidor se compila usando un Makefile. Para compilarlo solo hay que ejecutar: **make**

Con este comando se genera el ejecutable del servidor, llamado server.

Si se quieren borrar los ficheros generados durante la compilación, se puede usar: **make clean**

Para arrancar el servidor se ejecuta indicando el puerto: **./server -p <puerto>**

Por ejemplo: **./server -p 8080**

El cliente se ejecuta con Python, indicando la IP del servidor y el puerto:

python3 client.py -s <IP_servidor> -p <puerto>

Por ejemplo: **python3 client.py -s 127.0.0.1 -p 8080**

Cuando el cliente ya está arrancado, el usuario puede escribir los comandos por consola para usar el sistema.

Batería de pruebas

Para comprobar que la práctica funciona correctamente, se han hecho varias pruebas usando los scripts `app_cliente.py` y `test_pendientes.py`.

Si también se quiere probar el servidor de logs RPC, antes hay que iniciar `rpcbind` y después arrancar el servidor RPC.

Pruebas con `app_cliente.py`

- Primero se probó el ciclo normal de un usuario: registrarse, conectarse, consultar los usuarios conectados, desconectarse y darse de baja. El resultado fue correcto, ya que el cliente mostró mensajes como REGISTER OK, CONNECT OK, CONNECTED USERS OK, DISCONNECT OK y UNREGISTER OK.
- Probamos qué pasa al registrar dos veces el mismo usuario. El primer registro funcionó bien, pero el segundo devolvió USERNAME IN USE, osea que el servidor detecta correctamente que no puede haber nombres repetidos.
- Enviamos un mensaje a un usuario que no existe. Para ello, se registró y conectó un usuario emisor y después se intentó mandar un mensaje a un usuario no registrado. El resultado fue SEND FAIL, USER DOES NOT EXIST, así que el servidor controló bien ese error.
- Se comprobó el envío de mensajes a usuarios registrados pero desconectados. En este caso, el servidor aceptó el mensaje y devolvió SEND OK - MESSAGE 1, indicando que el mensaje se había guardado como pendiente.
- Se probó que el comando USERS solo pueda usarlo un usuario conectado. Si el usuario estaba registrado pero no conectado, el cliente muestra CONNECTED USERS FAIL, USER IS NOT CONNECTED.
- Intentamos conectar un usuario que no estaba registrado. El resultado fue CONNECT FAIL, USER DOES NOT EXIST, confirmando que el servidor comprueba si el usuario existe antes de permitir la conexión.

Además de estas pruebas más básicas también se han realizado otras donde se lanzan dos clientes al mismo tiempo.

- Una prueba fue el envío directo entre dos usuarios conectados para ver que se envían bien los mensajes cuando el receptor está conectado. Para ello se

registraron dos usuarios, se conectaron los dos y uno le envió un mensaje al otro. El usuario receptor recibió correctamente:

```
MESSAGE 1 FROM directo_a hola_directo
END
```

El usuario que envió el mensaje recibió la confirmación:
SEND MESSAGE 1 OK

- También se probó que se entregan los mensajes pendientes. Se mandó un mensaje a un usuario que estaba registrado pero no estaba conectado. El servidor guardó el mensaje y, cuando el receptor se conectó, lo recibió:

```
MESSAGE 1 FROM pend_emisor
Hola pendiente
END
```

Además, el emisor recibió la confirmación de que el mensaje se había entregado. Esto demuestra que los mensajes pendientes se guardan y se entregan más tarde cuando el usuario se conecta.

- Otra prueba fue enviar varios mensajes pendientes. Se mandaron tres mensajes al receptor mientras este estaba desconectado y cuando se conectó, recibió los tres mensajes en orden:

```
MESSAGE 1 FROM multi_emisor
mensaje1
END
```

```
MESSAGE 2 FROM multi_emisor
mensaje2
END
```

```
MESSAGE 3 FROM multi_emisor
mensaje3
END
```

- Por último, se probó que al dar de baja un usuario con UNREGISTER, también se eliminan sus mensajes pendientes. Para comprobarlo se envió un mensaje a un receptor desconectado y luego se dio de baja a ese receptor. Luego se volvió a registrar el mismo nombre de usuario y se conectó, este no recibió ningún mensaje antiguo, así que se confirma que los mensajes pendientes se eliminan bien al hacer UNREGISTER.

Problemas encontrados y conclusiones

Durante la práctica han aparecido varios problemas, sobre todo al trabajar con los mensajes pendientes, porque al principio si el receptor no estaba conectado el envío simplemente fallaba. Esto se solucionó guardando el mensaje en una lista de pendientes y revisando esa lista cuando el usuario hacía CONNECT.

Otro problema fue que el cliente no mostraba algunos mensajes porque el nombre de la operación no coincidía exactamente con el que mandaba el servidor. Se corrigieron las cadenas usadas en el protocolo, por ejemplo SEND MESSAGE y SEND MESS ACK.

Hubo también algunos problemas de compilación con RPC y de instalación de Flask. Para RPC se instaló libtirpc-dev y para Flask se decidió usar un entorno virtual con requirements.txt.

Como conclusión general, al principio la práctica parecía complicada por tener muchas partes diferentes, pero al ir haciéndola paso a paso se ha entendido mejor como funciona en general el sistema.